第三周：序列模型与语言模型

- 迁移学习

迁移学习是一种机器学习方法，其核心思想是将在一个任务（源任务）上学到的知识，应用到另一个相关任务（目标任务）上。迁移学习方法旨在解决深度学习中的训练难题，例如数据量不足或训练时间过长等问题。

  - **特征提取 (Feature Extraction)**：
    - 保留预训练模型（在源任务上训练好的模型）的大部分层（通常是前面用于提取特征的层），并冻结它们的权重，使其在新的任务上不再更新。
    - 仅用这些固定的层来提取新任务数据的特征。
    - 之后，在模型顶部添加一个新的分类器或回归层，并在目标数据集上训练这个新加的层。
  - **微调 (Fine-tuning)**：
    - 在特征提取的基础上，对预训练模型的**部分甚至全部参数**进行微调。
    - 这意味着在目标数据集上，不仅训练新加的层，还以一个非常小的学习率继续训练（更新）预训练模型的权重。

- 深度学习调参（Deep Learning Hyperparameter Tuning）

深度学习模型的损失函数通常是高度非凸的。这意味着其损失曲面存在多个局部最小值（local minima）和鞍点（saddle points），而不仅仅是一个全局最小值。

随着网络层数和神经元数量的增加，模型参数量巨大，损失曲面的维度也随之增加。在高维空间中，非凸优化问题会面临更复杂的挑战。

初始参数的选择对于训练过程至关重要。如果初始化不当，训练可能收敛到不好的局部最优解，甚至导致梯度消失或梯度爆炸，使训练失败。

传统的优化算法（如梯度下降）容易在训练过程中陷入损失曲面上的局部最小值。即使找到了一个局部最小值，它也不一定是全局最优解，导致模型性能不佳。

- 学习率与优化器

学习率决定了每次参数更新的步长。学习率过大，可能导致训练过程震荡甚至发散；学习率过小，则收敛速度会非常缓慢。

动态学习率 (Dynamic Learning Rate)指的是在训练过程中，根据预设的策略或模型表现动态调整学习率。**此外学习率衰减 (Learning rate decay)**是一种常见的动态学习率策略。随着训练的进行，学习率会逐步减小。这有助于模型在训练初期快速收敛，并在后期更精细地微调参数，避免震荡。

自适应学习率 (Adaptive Learning Rate)指的是一类更先进的优化算法，它们可以为每个参数自动调整其学习率。这意味着不同的参数可能有不同的学习率。

- **Adagrad**：为稀疏参数提供更大的学习率。
- **Adadelta**：是对 Adagrad 的改进，解决了其学习率持续衰减的问题。
- **RMSprop**：也是对 Adagrad 的改进，通过使用指数加权移动平均来解决学习率持续衰减的问题。
- **Adam**：虽然未在此页中列出，但它结合了 RMSprop 和 Momentum，是最受欢迎的自适应学习率优化器之一。

- Dropout

Dropout 是一种用于深度学习的正则化技术，旨在防止模型过拟合。它的工作原理是在训练过程中，以一定的概率随机地“丢弃”（即将其激活值设置为零）神经网络中的一部分神经元。这迫使网络不能过度依赖任何特定的神经元，从而学习到更具鲁棒性和泛化能力的特征。

在**测试阶段**，所有的神经元都会被激活。为了补偿训练时神经元数量的减少，需要对所有神经元的权重进行缩放。通常的做法是将训练好的权重乘以**丢弃概率的倒数**（例如，如果丢弃概率为 0.5，则权重乘以 0.5），以确保测试时的输出与训练时大致处于同一数量级。

- 词向量

词向量是一种将词语或短语从离散的符号表示（如独热编码）转换为连续的、稠密的实数向量的技术。在这些向量空间中，具有相似语义或语法功能的词语在向量空间中的距离也更近。

词向量是现代 NLP 模型的基础。它将离散的、无法直接进行数学运算的文本数据，转换成了计算机能够理解和处理的稠密向量，从而为后续的神经网络模型提供了高质量的输入。

- 序列到序列模型

Seq2Seq 模型是一种通用的神经网络框架，用于处理输入是序列、输出也是序列的任务。该模型通常由一个编码器（Encoder）**和一个**解码器（Decoder）组成。

  - **编码器**：读取输入序列（如一个句子），并将其压缩成一个固定长度的**上下文向量**（也叫“思想向量”或“上下文表示”）。这个向量包含了输入序列的全部信息。
  - **解码器**：以编码器生成的上下文向量为输入，逐步生成目标序列。在生成每个词时，解码器会利用上下文向量和之前生成的词语。

- RNN、LSTM、GRU
  - **RNN（Recurrent Neural Network）**：最基本的循环神经网络。它通过一个**隐藏状态**（Hidden State）来捕获序列历史信息。在处理序列的每个时间步时，RNN 会将当前输入和上一个时间步的隐藏状态作为输入，并生成新的隐藏状态和输出。
  - **LSTM（Long Short-Term Memory）**：RNN 的改进版，旨在解决 RNN 在处理长序列时出现的**梯度消失**和**梯度爆炸**问题。LSTM 引入了特殊的**门控机制**（输入门、遗忘门、输出门），来精确控制信息在隐藏状态中的流动，从而能够更好地捕获长距离依赖关系。
  - **GRU（Gated Recurrent Unit）**：LSTM 的简化版。它也使用门控机制来解决长距离依赖问题，但只用了两个门（更新门和重置门），参数更少，计算更高效，但在很多任务上性能与 LSTM 相当。

- 注意力机制

注意力机制是一种让模型在处理序列时，能够动态地关注（或“分配权重”）输入序列中不同部分的技术。它解决了 Seq2Seq 模型中编码器将所有信息压缩到固定长度上下文向量时，可能导致的信息丢失问题，尤其是在长序列任务上。

在解码器生成每个输出词时，注意力机制会计算一个**注意力分数**，来衡量输入序列中每个词对当前输出词的重要性。然后，它将这些分数作为权重，对所有输入词的编码表示进行加权求和，生成一个新的**上下文向量**。这个新的上下文向量包含了对当前输出最有用的信息，而不是一个固定不变的向量。

- Transformer

Transformer 是一种完全基于**注意力机制**的深度学习模型，使用自注意力机制（Self-Attention）来处理序列。

  - **自注意力（Self-Attention）**：Transformer 最核心的组件。它使得模型能够同时处理序列中的所有词，并计算每个词与其他所有词之间的关系。这意味着，它可以一次性地捕

获序列中的长距离依赖关系，而不像 RNN 那样需要按顺序处理。

- **位置编码（Positional Encoding）**：由于 Transformer 没有 RNN 那样的循环结构，它失去了处理序列顺序的能力。为了弥补这一点，模型在输入中加入了位置编码，来提供序列中每个词的相对或绝对位置信息。
- **编码器 - 解码器结构**：Transformer 同样采用了编码器 - 解码器架构，但每个部分都由多个相同的层堆叠而成。